

Ares Laptop — Disaster Recovery Runbook

Tags: #runbook #disaster-recovery #ares #restic #ops **Created:** 2026-07-20 **Scope:** Total loss of the Ares admin/jump laptop → back to a working box.

The next hardware death should be fast and boring. This is the do-this-next-time procedure, ordered start to finish. It was written from the 2026-07 recovery of the old Ares (an ~8-yr-old laptop that died mid-session with a motherboard power-delivery failure) onto a **Dell Latitude 3580** (service tag 5HWCNJ2).

Full narrative + evidence: [ares-recovery-context.md](#) (Part 1 = claude.ai chat reconstruction; Part 1B = machine-history ground truth). The restic mechanism itself is also documented in [../vault/Runbook/Ares-Backup-Restore.md](#) — this runbook is the wider “hardware is dead, rebuild everything” superset.

Ares is only the admin box, not a cluster member. When it dies the cluster keeps running — DNS, Proxmox, OPNsense, backups all stay up. There is no production outage clock. Work the steps in order; don’t rush hardware.

Key facts to keep this runbook accurate (verbatim — do not paraphrase):

Item	Value
Backup tool	restic (NOT PBS — see §5)
Repo	sftp:randy:/mnt/bulk/backups/ares (encrypted)
Backup host	Randy — 192.168.10.187, bulk ZFS pool
randy SSH alias	root@192.168.10.187 (in ~/.ssh/config) — resolves to root , which is why SFTP can read the repo
Repo dir perms	root-owned, mode 700 — only root can read it
Passphrase (on-disk)	~/.config/restic/ares-randy.pass (mode 600)
Passphrase (offsite)	Vaultwarden entry Ares restic backup -> Randy (repo password) (pve3 LXC 102)
Hostname	ares — reuse it (see §2)
Ares IP	192.168.10.199 (mgmt VLAN 1)

§0 — Pre-disaster checklist (the prevention layer)

Do these while Ares is healthy. Each one directly collapses a step that cost real time in the last recovery. This is the highest-leverage section in the document.

- ☐ **Offline copy of the three secrets that gate everything else.** Put the restic passphrase **and** the SSH private keys (~/.ssh/id_ed25519*) **and** the Ansible vault key somewhere that survives the laptop dying: a phone password manager, a printed copy in a safe, or the salvaged NVMe in an enclosure (below). *Why:* last time the passphrase lived only in Vaultwarden, and reaching it needed a from-scratch nginx TLS proxy on a machine with nothing installed (§4). An offline copy collapses §4 to a copy-paste.
- ☐ **Keep a USB 3.2 NVMe M-key enclosure in the kit permanently** (ORICO / UGREEN, ~\$15–20). *Why:* it is the fastest path to the old drive’s secrets and the ultimate fallback source of truth. The salvaged 970 EVO Plus is currently unreadable precisely because there’s no enclosure on hand.
- ☐ **The backup mechanism is documented and unambiguous:** restic at sftp:randy:/mnt/bulk/backups/ares, repo dir is **root-owned mode 700**, so restic must run **as root** (the **randy** alias already is root). It is **not PBS**. See §5. *Why:* last time recovery started down the PBS / proxmox-backup-client path and lost time before discovering it was restic.
- ☐ **A working Debian DVD-1 ISO is on hand** (not just netinst) with a way to flash it (Rufus). *Why:* the Intel Wireless-AC card needs non-free firmware the netinst can’t fetch without a network (§3).

- **The backup is monitored and freshness is trusted.** The AresBackupStale Grafana rule (>26h → Discord) must be live and believed. *Why:* verify snapshot age FIRST every time (§1); don't assume the backup is current.
-

§1 — Confirm the backup is good BEFORE touching hardware

Prove you have a restorable, recent, intact snapshot **before** buying or wiping anything. If this step fails, fix the backup story first — everything downstream assumes it.

From any machine that can reach Randy and has the passphrase (this can be done before the new laptop even exists — e.g. from another box or a phone SSH client):

```
ssh randy 'restic -r /mnt/bulk/backups/ares snapshots'
```

Lists every snapshot with ID + timestamp. Run **on Randy as root** (the `randy` alias is root, and the repo is root-only), so you avoid the SFTP permission trap entirely for a read-only check. Confirm the newest snapshot is recent — the last real recovery found the newest at **2026-07-17 04:03, 110.212 GiB**.

```
ssh randy 'restic -r /mnt/bulk/backups/ares check'
```

Verifies repo structure and metadata are intact (fast). This is *not* a full data re-read — for that, `restic check --read-data` pulls back all ~110 GiB; skip it routinely (ZFS checksums on bulk already guard bit-rot) but consider it once before you delete any staging copy.

Trap from last time: don't judge freshness by reading the backup *log* — and especially not the log *inside a restored copy*, which can be a mid-run snapshot that ends at “backup starting” with no completion line. Trust `restic snapshots` against the **live** repo, not a log artifact.

If a stale lock blocks you: `restic unlock` clears an abandoned lock left by the dead machine.

§2 — Replacement hardware selection

Check interfaces and RAM **before you buy or commit a drive** — the last stopgap laptop had two nasty surprises that a spec check would have caught.

Interface / RAM checklist (verify against the actual model's service manual):

- **Drive interface: NVMe vs SATA, and is there an M.2 NVMe slot at all?** The Latitude 3580 has **no M.2 NVMe slot** — only a 2.5” SATA bay + WLAN/WWAN. Its RWMDf caddy carries **M.2 SATA only**, not NVMe/PCIe. The salvaged 1TB 970 EVO Plus (NVMe) therefore **cannot be used internally** — a WD Black 2.5” SATA drive went in the bay instead. Confirm the slot type before planning to reuse any drive.
- **RAM is socketed, not soldered.** (A ThinkPad T14 Gen 2 was rejected for soldered RAM.) Socketed = upgradable and repairable.
- Enough of everything for a jump box: the old Ares had 32GB RAM.

Reuse the hostname ares. This is a rule, not a preference. Keeping the name means you do **not** have to touch:

- Ansible inventory
- Pi-hole DNS records
- DHCP MAC reservations
- Headscale node entries
- every script that hardcodes `ares`

On any machine that had SSH'd to the *old* Ares, clear its stale host key so you don't hit “host key verification failed” against the new box:

```
ssh-keygen -R ares
ssh-keygen -R 192.168.10.199
```

Removes the old fingerprint for the name and the IP from `known_hosts`. (This exact class of stale-key failure once masqueraded as an “auth failure” against the EX3400 — see EX3400-SSH-Auth-Failure-RCA.md.)

§3 — Install Debian

Target: **Debian 13 Trixie + KDE Plasma 6** (better 7th-gen Intel firmware support than Debian 12).

Media: - Use the full **DVD-1 ISO**, *not* netinst. *Why:* the Intel Wireless-AC card needs non-free firmware; netinst assumes a working network to fetch it, which you don’t have yet. The DVD carries firmware + the full KDE set offline. - Flash with **Rufus in DD mode** (not ISO mode). *Why:* ISO mode rewrites the boot sector and corrupts Debian’s hybrid ISO; DD writes it byte-for-byte. Target **GPT / UEFI**.

Installer: - At software selection, **UNCHECK GNOME, CHECK KDE Plasma** — Debian defaults to GNOME and it’s easy to blow past. - Set hostname **ares** (see §2).

§3.1 — UEFI / Secure-Boot / GRUB-NVRAM recovery (expect this on Dell)

GRUB installs fine but the machine boots to firmware with no `debian` entry — the EFI files are on the ESP but the NVRAM boot variable didn’t stick (it clears when the USB is pulled and the firmware reshuffles boot order). Two fixes; the first is what worked.

Fix A — add the boot entry by hand in firmware (what worked): - In the Dell firmware boot menu, add a boot option pointing at `\EFI\debian\shimx64.efi` (fallback `\EFI\debian\grubx64.efi`), name it `debian`, and move it to the **top** of the boot order. - **Secure Boot:** Dell firmware can silently re-enable it after changes, and then shim refuses to load. Re-confirm **Secure Boot OFF** under **F2 → Boot Configuration**.

Fix B — rescue mode (same install USB → Advanced → Rescue):

```
grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=debian --recheck
update-grub
efibootmgr -v
```

Reinstalls GRUB’s EFI files, rewrites the NVRAM boot entry named `debian`, and prints the boot order so you can confirm it took. Remove leftover Windows entries with:

```
efibootmgr -b <NNNN> -B
```

Deletes boot entry number `NNNN` (read the number from `efibootmgr -v` output).

§4 — Retrieve the restic passphrase

You need the repo passphrase before you can restore (§5). Take the fast path if you prepared §0; the Vaultwarden path is the fallback only.

§4.1 — Fast path (use this if §0 was done)

- **Offline copy:** phone password manager, printed copy, or wherever you stashed it. Copy it and go straight to §5.
- **Salvaged NVMe via enclosure:** put the old 970 EVO Plus in the USB NVMe enclosure, mount it read-only, and read the passphrase directly from `~/.config/restic/ares-randy.pass` (the SSH keys and Ansible vault key are right there too). This makes the whole Vaultwarden detour unnecessary.

§4.2 — Fallback: pull it from Vaultwarden

Vaultwarden runs on pve3 (LXC 102) and survives Ares dying, but it needs HTTPS and can't be hit directly from a fresh laptop. Two ways in:

Option 1 — read the secret straight out of the container over SSH (fastest; skips the browser entirely). If the entry you need is stored as a container env var / file, `pct exec` into it from a Proxmox node:

```
ssh root@192.168.10.201 "pct exec 102 -- grep ADMIN_TOKEN /etc/vaultwarden.env"
```

192.168.10.201 = pve3, 102 = the Vaultwarden LXC. (This exact command pulled the Vaultwarden admin token during the last recovery. The restic passphrase is a *vault item*, not an env var, so this gets you the admin token / into the box — you may still need the UI or `bw` CLI to read the item itself.)

Option 2 — stand up a throwaway TLS proxy and use the web UI (what was done last time). Vaultwarden speaks plain HTTP on :8080; the browser wants HTTPS. Tunnel to it, then front it with a self-signed TLS proxy:

```
ss -tlnp | grep -E '8080|8443'          # confirm the tunnel/listener is up
sudo apt install -y nginx openssl      # neither is on a fresh box
sudo openssl req -x509 -newkey rsa:2048 -nodes -days 30 \
    -keyout /etc/ssl/private/vw.key -out /etc/ssl/certs/vw.crt -subj "/CN=localhost"
```

First line verifies the SSH local-forward to :8080 is listening. The `openssl` line mints a 30-day self-signed cert for `localhost`. Then add an `nginx` server block listening 8443 `ssl` that proxies to `127.0.0.1:8080`, validate and load it:

```
sudo nginx -t && sudo systemctl restart nginx
```

`nginx -t` checks the config before restart. Open `https://localhost:8443` in Firefox, accept the self-signed cert, log in, and copy the entry **Ares restic backup** -> **Randy (repo password)**.

After recovery, close this gap for good (§0): get the passphrase into an offline store so §4.2 never happens again. During the last recovery the restic entry was *added* to Vaultwarden as remediation — also keep a copy off-cluster.

§5 — Restore with restic

This is **restic**, not PBS. Do **not** reach for `proxmox-backup-client / .pxar / host/ares/<timestamp>` — that path was a dead end last time.

Repo reality: `/mnt/bulk/backups/ares` on Randy is **root-owned, mode 700**. Only root can read it. The trick that makes this painless: the `randy` SSH alias in `~/.ssh/config` already points at `root@192.168.10.187`, so `sftp:randy:` is an SFTP session *as root* and can read the repo — you do **not** have to log into Randy and run `restic` there. (For a read-only `snapshots/check` it's still fine to run on Randy directly, as in §1.)

Step 1 — get restic on the fresh box (it won't be installed):

```
sudo apt install -y restic
```

Installs the `restic` binary. (The healthy Ares also keeps a static binary at `~/.local/bin/restic`; either works.)

Step 2 — point restic at the repo and unlock it:

```
export RESTIC_REPOSITORY="sftp:randy:/mnt/bulk/backups/ares"
export RESTIC_PASSWORD_FILE=~/.config/restic/ares-randy.pass # if you restored the file
# - or, with the passphrase from §4, avoid putting it in shell history:
```

```
# read -rs RESTIC_PASSWORD; export RESTIC_PASSWORD
restic snapshots
```

RESTIC_REPOSITORY selects the repo over SFTP-as-root; the password is supplied by file (preferred) or env. `restic snapshots` confirms you can actually read it and shows the snapshot you're about to restore.

Security note (learned the hard way): last time the passphrase was inlined as `export RESTIC_PASSWORD='...'`, which left it in **plaintext** in `~/.bash_history`. Prefer `RESTIC_PASSWORD_FILE`, or `read -rs` so it never hits history. If you do inline it, scrub the line from `~/.bash_history` afterward.

Step 3 — restore to staging, NEVER straight onto live /home:

```
restic restore latest --target ~/restore
```

Restores the newest snapshot into `~/restore` on the laptop (not onto live `/home`, which KDE has dotfiles open in — overwriting live is unpredictable). This runs from the laptop over SFTP-as-root and lands directly in the staging dir; no Randy-side staging or `rsync` is needed. To pull just one tree, add `--include /home/machismo/Home-Lab`.

Step 4 — inspect, then selectively reinstate. With the old home sitting under `~/restore/home/machismo/`, copy back what you need — dotfiles, `~/.ssh/` keys (check perms: 600 on private keys), the Home-Lab repo, `~/.config/restic/`, and the backup script/timer. Do it deliberately, not with a blind overwrite.

Step 5 — repopulate excluded content. Upstream git submodules (`claude-desktop-debian`, `pacextractor`, `spreadtrum_flash`, `CVE-2022-38694_unlock_bootloader`) and `caches/venvs` were excluded from the backup to save space:

```
git -C ~/dotfiles submodule update --init --recursive
```

Re-clones the excluded submodules from their own remotes. Rebuild Python `venvs` from their `requirements.txt` as needed.

Step 6 — reinstate the backup tooling immediately (this is what stopped backups last time — the rebuild wiped `restic`, `ares-backup.sh`, `~/.config/restic/`, and all user timers, so `AresBackupStale` fired). Restore the `restic` binary to `~/.local/bin/`, `ares-backup.sh`, and `~/.config/restic/`, then enable the timer and run once:

```
systemctl --user enable --now ares-backup.timer
systemctl --user list-timers ares-backup.timer
```

Enables the daily 04:00 backup timer and shows its next run, so the new box is protected again the same night. Full detail: [../vault/Runbook/Ares-Backup-Restore.md](#).

Clean up staging once reinstated: `~/restore` is temporarily excluded in `~/.config/restic/excludes.txt` so it isn't double-captured. Remove it (and the exclude line) once you're confident everything is reinstated.

§6 — Rebuild the toolchain (Claude Code)

Install Claude Code via the official installer:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Installs the `claude` CLI into `~/.local/bin` (last recovery landed v2.1.215).

claude: command not found — the PATH fix. `~/.local/bin` isn't on `PATH` for interactive non-login shells: the export lived only in `~/.profile` (login shells) and was missing from `~/.bashrc` (the normal desktop-terminal case). Add it to `~/.bashrc`:

```
grep -n 'local/bin' ~/.bashrc ~/.profile # see what's already there
printf '\nexport PATH="$HOME/.local/bin:$PATH"\n' >> ~/.bashrc # append the export
source ~/.bashrc && which claude && claude --version # confirm it resolves
```

`grep` shows whether the export exists; `printf` appends it cleanly (preferred over `echo` for predictable newlines); `source` reloads the shell and the last two commands confirm `claude` is now found.

OPEN ISSUE — Claude account (runbook can't fix this, only flag it): on the last rebuild, Claude login landed in an **empty account with a payment prompt despite an active subscription** — likely a duplicate account from a different auth method. The account ID is **not** usable as a CLI credential. Resolve it with Anthropic support at support.claude.com; there is no local workaround.

§7 — Post-restore verification checklist

Prove the rebuilt Ares is fully back in its admin/jump role. Work top to bottom.

- ☐ **SSH keys intact:** `ls -l ~/.ssh` — private keys mode 600; `randy`, `quarkylab`, etc. aliases present in `~/.ssh/config`.
- ☐ **Reach Randy as root:** `ssh randy 'hostname && zpool list'` — confirms the alias, the key, and Randy's pools (`bulk`, `datastore`) are healthy.
- ☐ **SSH into each cluster node:** `pve2 .204`, `pve3 .201`, `pve4 .202`, `pve5 .203`, plus `pve1 .193`, `Quarky-Lab .179`, `Jarvis .31`. Each should accept the key without a password.
- ☐ **Proxmox reachable:** open a node UI, e.g. <https://192.168.10.201:8006>.
- ☐ **DNS resolves via Pi-hole:** `dig @192.168.10.177 kylemason.org +short` (primary) and `@192.168.10.178` (secondary) both answer.
- ☐ **Ansible sees the fleet (dry run, no changes):** `ansible all -m ping` then `ansible-playbook <play> --check --diff`. Confirms the inventory, the vault key, and SSH all work end-to-end.
- ☐ **Backups running again:** `systemctl --user list-timers ares-backup.timer` shows a next run; after the first run, `restic snapshots` shows a fresh-dated snapshot and `AresBackupStale` clears in Discord.
- ☐ **git working trees clean:** `git -C ~/dotfiles status` and `git -C ~/Home-Lab status` — restored, on the right branch, submodules initialised.
- ☐ **Secrets hygiene:** confirm the `restic` passphrase is in an offline store (§0) and scrub any plaintext secret from `~/.bash_history` (§5 security note).

Appendix — Lessons that shaped this runbook

1. **Reuse the hostname** — it kept Ansible/DNS/DHCP/Headscale/scripts untouched.
2. **Keep the secrets offline** — the single biggest avoidable cost was the Vaultwarden-only passphrase and the from-scratch TLS proxy to reach it.
3. **Check the drive interface before buying/reusing** — NVMe vs SATA, M.2 slot presence. The salvaged NVMe was dead-on-arrival for a SATA-only 3580.
4. **Keep a USB NVMe enclosure in the kit** — the fallback source of truth.
5. **DVD ISO + Rufus DD** for offline-firmware laptops, and expect the Dell UEFI NVRAM / Secure-Boot boot-entry dance.
6. **It's restic, not PBS** — `sftp:randy:/mnt/bulk/backups/ares`, root-owned 700, run as root. Enumerate ZFS pools before hunting for a repo.
7. **Verify snapshot freshness first, every time** — against the live repo, not a log artifact.
8. **Restore to staging, inspect, then reinstate** — never straight onto live /home.